Finance Research Letters 54 (2023) 103809

Contents lists available at ScienceDirect

Finance Research Letters

journal homepage: www.elsevier.com/locate/frl

Deep learning and technical analysis in cryptocurrency market

Stéphane Goutte [b,e], Hoang-Viet Le [a,b,*], Fei Liu [c], Hans-Jörg von Mettenheim [c,a,d]

[a] Keynum Investments, France
[b] Université Paris Saclay, UMI SOURCE, IRD, UVSQ, France
[c] IPAG Business School, France
[d] Oxford-Man Institute of Quantitative Finance, United Kingdom
[e] Paris School of Business, 59 Rue Nationale, 75013, Paris, France

ARTICLE INFO

Keywords:
Bitcoin
Technical analysis
Machine learning
Deep learning
Convolutional neural networks
Recurrent neural network

ABSTRACT

A large number of modern practices in financial forecasting rely on technical analysis, which involves several heuristics techniques of price charts visual pattern recognition as well as other technical indicators. In this study, we aim to investigate the potential use of those technical information (candlestick information as well as technical indicators) as inputs for machine learning models, especially the state-of-the-art deep learning algorithms, to generate trading signals. To properly address this problem, empirical research is conducted which applies several machine learning methods to 5 years of Bitcoin hourly data from 2017 to 2022. From the result of our study, we confirm the potential of trading strategies using machine learning approaches. We also find that among several machine learning models, deep learning models, specifically the recurrent neural networks, tend to outperform the others in time-series prediction.

1. Introduction

Even since the success of Google's AlphaGo against the Go World Champion in 2016, the keywords AI (Artificial Intelligence), machine learning, and deep learning have become the buzzwords that keep being mentioned in almost every industry. As a matter of fact, machine learning has already been slowly yet steadily transforming your everyday activities.

The finance industry, as expected, is one of the pioneers to incorporate AI technologies into its ecosystems. In recent years, more and more studies have been published that were based on the application of machine learning models on forecasting time series and many of them point out the relatively better performances of newer machine learning models compared to classical time series forecasting techniques (Rodriguez and Rodriguez, 2004; Cosenza et al., 2021; Yun et al., 2021). Additionally, the widespread and easy-to-access automated electronic trading systems coupled with increasing demand for better alpha keep forcing trading practitioners and researchers to put more effort into searching for better and more sophisticated models. Not until recently, the technique of deep learning (DL), a subset of machine learning which involves the application of neural networks, started to emerge as one of the best-performing techniques within the field of machine learning in a vast majority of implementation areas. It is no exception in the case of financial time series forecasting as a growing number of deep-learning-based models of prediction have been introduced in several conferences and journals in recent years. Gu et al. (2020) used neural networks to forecast stock prices from the US market to find out their superiority over other models such as generalized linear models, principal components regressions, and regression trees (Gu et al., 2020). Zhang et al. (2021) utilize recurrent neural networks, specifically long short-term memory (LSTM) networks, to predict stock trends and report significant positive returns (Zhang et al., 2021). The work of Bianchi et al. (2021), on

* Corresponding author at: Keynum Investments, France.
E-mail address: viet.le@keynum.fr (H.-V. Le).

https://doi.org/10.1016/j.frl.2023.103809
Received 27 December 2022; Received in revised form 25 February 2023; Accepted 15 March 2023
Available online 20 March 2023
1544-6123/© 2023 Elsevier Inc. All rights reserved.

Table 1
Trade data table.

	Field name	Type	Description
1	timestamp	datetime	The moment at which the transaction occurs, with precision to the millisecond.
2	symbol	string	Ticker representing trading instrument.
3	side	string	The side of the trade, either Buy or Sell.
4	size	integer	The transaction size of the trade
5	price	float	The price that the trade was executed

Table 2
The preprocessed trade data.

	Field name	Type	Description
1	timestamp	datetime	The beginning time of the given period.
2	symbol	string	Ticker representing trading instrument, we only concern the ticker BTCUSD.
3	open	float	The latest price that happened before the beginning of the period
4	high	float	The maximum price that happened during the period
5	low	float	The minimum price that happened during the period
6	close	float	The latest price that happened before the end of the period
7	trades	integer	The total number of transactions that happened during the period
8	volume	float	The total volume of currency that was traded during the period

the other hand, focuses on forecasting bond excess returns and also report significant economic gains. Some other studies target other financial assets such as cryptocurrencies and also often point out the edge of the neural networks model on forecasting future price movements (Anghel, 2021).

In this paper, we tried to put into practice those state-of-the-art deep learning models along with other popular machine learning models to see how well they perform in finding trading signals using the same inputs that were used in traditional technical analysis. While most of the other research on this topic focuses on the traditional stock markets, this paper chooses the cryptocurrency market, specifically the Bitcoin market, as the target of research.

2. Dataset

2.1. Data source

Due to the borderless nature of the cryptocurrency market, the prices of cryptocurrencies are not the same everywhere but vary between exchanges. In this research, we chose the Binance exchange as it is the largest cryptocurrency exchange based on its trading volume as of August 2022. The Binance exchange publicly offers historical data dating back to 2017. Transaction details are stored continuously and are accessible via its API[1]. The exchange houses several sets of high-frequency data including limit order quotes and trading logs (or market order logs). Due to the goal of our analysis, only the trading logs (or trade data) are needed. Table 1 lists out all the fields of the trading logs table and their meanings.

The trade data table records every single transaction that had taken place. Each row refers to a transaction processed by the Binance exchange and obviously, it is possible that many transactions can happen at the same time.

As every transaction is recorded in this table, the total number of transactions provided during our chosen period from the end of 2017 until August 2022 is up to a few billion. This is an enormous amount of data and is required to be processed for further analysis.

2.2. Data preprocessing

The first step of data preprocessing is to gather all of the needed data values from different data sources into a single collection of databases. In this study, as all of our data come from the historical trading database of Binance, there is no need for other sources of data. However, if any other types of alternative data are considered such as order book data, sentimental or fundamental data, etc., they should be included in this step. In this study, during these steps, several trivial data such as the ID of a single trade, type of order, etc. will also be removed. In the end, all of the trades that happened in every hour are resampled and grouped by their timestamps to create a new table of time series data with the following header: timestamps, open, high, low, close, number of trades, and volume (see Table 2).

After the resampling procedure, the length of our dataset is reduced to only 43,463 data points. For the purpose of training our machine learning models, the data will be divided into 3 parts: training set, validation set, and test set with the proportion of 50%, 25%, and 25% respectively based on chronological order as described in Fig. 1.

[1] https://api.binance.com/api/v3/klines

Fig. 1. Bitcoin dataset.

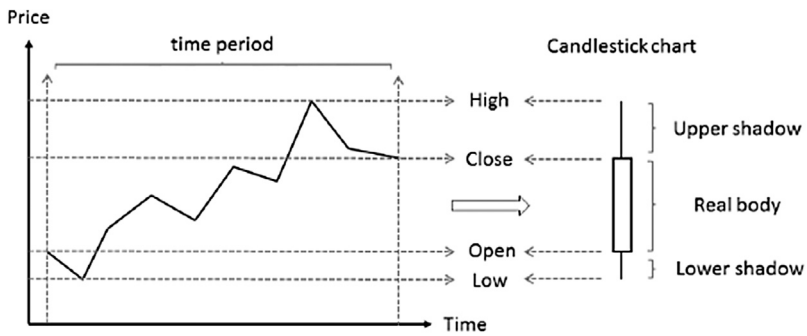

Fig. 2. Candle stick example.
Source: (Wang et al., 2015)

3. Methodology

3.1. Feature engineering

Candlestick Data

Generally, the asset price data of a specific time period can be represented by four features: the opening price (the price at the beginning of the period), closing price (price at the end of the period), high price (the maximum price during the period), and low price (the minimum over the interval). However in technical analysis, the candlestick can be visually encoded in a different way (Fig. 2): the candle body denotes the opening and closing prices, and the lines protruding from the bar (the candle's shadow) denote the extrema over the interval. Finally, the color of the bar implies the relative return of the period: a white (sometimes green) candle denotes a positive return over the interval (closing price > opening price) and a black (sometimes red) candle denotes a negative return (closing price < opening price). This alternative representation of the candlestick data may connect closer to how technical analyst visualize their charts and candlestick pattern and has been used by Ghoshal and Roberts (2020).

The specific formulas to transform the price variables into a candlestick are as follows:

$$Color = \begin{cases} White & \textbf{if} \quad Close - Open > 0 \\ Black & \textbf{if} \quad Close - Open < 0 \end{cases} \tag{1}$$

$$Candle\ Body = |Close - Open| \tag{2}$$

$$Upper\ Shadow = \begin{cases} (High - Close) & \textbf{if} \quad Close - Open > 0 \\ (High - Open) & \textbf{if} \quad Close - Open < 0 \end{cases} \tag{3}$$

Table 3
Descriptive statistics of all variables.

	Count	Mean	Std	Min	Max
open	46,904.00	19,448.43	16,744.98	2,870.90	68,635.12
high	46,904.00	19,560.10	16,840.24	2,950.00	69,000.00
low	46,904.00	19,329.50	16,644.04	2,817.00	68,451.19
close	46,904.00	19,448.68	16,744.84	2,919.00	68,633.69
volume	46,904.00	2,849.62	3,754.16	–	137,207.20
trades	46,904.00	51,179.13	74,852.01	–	1,330,791.00
SMA_15	46,904.00	19,446.84	16,742.70	3,197.87	67,886.42
SMA_20	46,904.00	19,446.19	16,742.00	3,204.75	67,621.23
SMA_25	46,904.00	19,445.54	16,741.29	3,213.43	67,541.40
SMA_30	46,904.00	19,444.89	16,740.58	3,215.02	67,416.47
RSI_15	46,904.00	50.69	11.96	4.78	96.04
RSI_20	46,904.00	50.64	10.61	6.85	94.16
RSI_25	46,904.00	50.61	9.65	8.87	92.33
RSI_30	46,904.00	50.59	8.93	10.79	90.60
MACD	46,904.00	1.83	231.09	(2,360.95)	1,887.22
WR	46,904.00	(46.76)	25.90	(100.00)	–
SO	46,904.00	53.24	25.90	–	100.00
MFI	46,904.00	50.49	16.78	–	100.00

$$Lower\ Shadow = \begin{cases} (Open - Low) & \textbf{if} \quad Close - Open > 0 \\ (Close - Low) & \textbf{if} \quad Close - Open < 0 \end{cases} \tag{4}$$

Technical Indicators

Despite its popularity, in practice, candlestick patterns are often used in combination with several other qualitative indicators such as support and resistance, Fibonacci retracement (the use of horizontal lines to indicate areas where an asset may experience support or resistance), etc. or even popular statistical indicators like the moving average, etc. In our study, aside from the raw features mentioned in the previous section, we also introduced an extended dataset that includes several popular technical analysis indicators to reflect the actual practice of technical analysis. The results from both the "raw" dataset and the "extended" data set will also be compared to see the effectiveness of technical indicators in the model (see Table 3).

In the extended dataset, we include simple moving average (SMA) and relative strength index (RSI) of multiple time steps, and other more complex indicators such as the moving average convergence/divergence oscillator (MACD), the Williams percent range (Williams %R), the stochastic oscillator, and the money flow index (MFI).

Descriptions and formulas for each technical analysis indicator used in our research are as follows:

- **Simple Moving Average (SMA):**
  The SMA is an average of closing price for any given period. Averaging window slides over time.

$$SMA(M, n) = \sum_{k=i+n}^{i+n} \frac{M(k)}{n} \tag{5}$$

- **Relative Strength Index (RSI):**
  RSI is a technical momentum indicator that shows the historical strength or weakness of the financial asset in a recent period

$$RSI = 100 - \frac{100}{1 + \frac{\text{avg gain}}{\text{avg loss}}} \tag{6}$$

- **Moving Average Convergence/Divergence line (MACD line):**
  MACD line is a momentum indicator that shows the relationship between series.

$$MACD_{line}(t) = (EMA(M, t, \tau_1) - EMA(M, t, \tau_2)) \tag{7}$$

- **MACD histogram:**
  MACD histogram analyzes the relationship between MACD Line and another trigger signal and provides buy and sell signal.

$$MACD_{hist} = MACD_{line}(t) - EMA(MACD_{line}, t, \tau_3) \tag{8}$$

- **Williams percent range (Williams%R)**
  William %R measures overbought and oversold line of market.

$$R = \frac{max(high) - close}{max(high) - min(low)} \times -100 \tag{9}$$

Table 4
Data labeling.

Dataset	Proportion	Long	Short
Training	50.00%	51.29%	48.71%
Validation	25.00%	51.54%	48.46%
Test	25.00%	49.72%	50.28%

- **Stochastic Oscillator:**

$$\begin{aligned} K &= \frac{close - min(low)}{max(high) - min(low)} * 100 \\ D &= SMA(K, 3) \\ SO &= K - D \end{aligned} \tag{10}$$

- **Money Flow Index (MFI).**
  MFI is a momentum indicator that uses volume data. It measures the inflow and outflow of money over any period of time.

$$\begin{aligned} TypicalPrice(t) &= \frac{high(t) + low(t) + close(t)}{3} \\ RawMoneyFlow &= TypicalPrice(t) \times Volume(t) \\ MFR(i) &= \frac{(\sum_{i-\tau}^{i} PositiveMoneyFlow)}{(\sum_{i-\tau}^{i} NegativeMoneyFlow)} \\ MFI(i) &= 100 - 100/(1 + MFR(i)) \end{aligned} \tag{11}$$

3.2. Data labeling

For financial time series problems, especially when it relates to a trading system, there is no fixed answer for the data labeling. If the research wants to predict the future price then it should be a regression problem and the future price of the next data point should be the label. But for a trading signal generating system, the label can be binary ($[Buy, Not\ Buy]$) or multi-class like $[Buy, Sell, Hold]$ depending on the preference of the researchers as well. In our study, our labels are chosen to be binary ($[Buy, Not\ Buy]$ or $[Sell, Not\ Sell]$) which are similar to trading signals used in technical analysis (specifically in the case of candle stick patterns where you have bullish patterns and bearish patterns). Furthermore, this choice of two separate systems for long and short also make it easier the inclusion the prediction with the threshold where we need to search for the best threshold using the validation set. The dimension for the search will be only one with two separate long and short systems instead of two for systems that are both long and short. Also with the threshold included, a both long and short system is more similar to a $[Buy, Sell, Hold]$ system where it does nothing when the prediction is below the threshold to long and above the threshold to short. More details about this procedure will be explained in a later section. As there is no threshold for the return used, basically we have the signal for that positive return will be labeled as the signal for buy and vice versa. The ratios of the labels are shown in Table 4.

The prediction horizon is set to the next hour or a candlestick and we set up our trading experiment as follows: For every trading signal, 1 dollar worth of BTC is either shorted or longed based on the signal. The position will be closed after 1 h unless there is a subsequent positive signal. The lookback period is also set at 24 h which means that historical data of each feature for the last 24 h will be used as inputs to forecasting the trading decision.

3.3. Model specifications

In this paper, seven popular machine learning algorithms were used which is simple Logistic regression, two decision tree-based gradient boosting models (XGBoost and LightGBM), and four neural network models (MLP, GRU, LSTM, and CNN).

For the logistic regression model, we choose the logistic regression algorithm provided by the Scikit-learn library using the default specification with the L2 regularization to prevent overfitting. For both XGBoost and LightGBM, we used their official Python libraries. For these two algorithms, we use 100 trees as it is a popular parameter used in several pieces of research (Basak et al., 2019; Nobre and Neves, 2019; Zolotareva, 2021; Hancock and Khoshgoftaar, 2020) and all others were set as default.

Regarding neural network models, in this study, all of our models are built and trained using the Python library Keras running on top of the TensorFlow framework. The architecture of our neural networks will generally follow the work of Ghoshal and Roberts (2020). In his work, he built an MLP model and extended it to other deep-learning models based on it. The architecture of the MLP model is as in Table 5

For the 2 recurrent neural network models, LSTM and GRU, to keep the model to be comparable with the MLP model, the new recurrent layer with only 8 neurons will be added before the 2 fully-connected layers of the MLP. Table 6 illustrates the architecture of the LSTM and GRU model.

Finally for the CNN, instead of the recurrent layers like LSTM or GRU, two convolutional layers are added along with two max-pooling layers and 0.5 dropouts each. Each of the convolutional layers has 8 neurons, similar to the recurrent neural network architecture (see Table 7).

Table 5
MLP architecture.

#	Layer	Units	Activation function	Dropout
0	Input	–	–	–
1	Dense	64	ReLU	0.5
2	Dense	64	ReLU	0.5
3	Dense	1	Sigmoid	–

Table 6
LSTM/GRU architecture.

#	Layer	Units	Activation function	Dropout
0	Input	–	–	–
1	LSTM/GRU	8	ReLU	0.5
2	Dense	64	ReLU	0.5
3	Dense	64	ReLU	0.5
4	Dense	1	Sigmoid	–

Table 7
CNN architecture.

#	Layer	Units	Activation function	Dropout	Filter size	Padding
0	Input	–	–	–	–	–
1	Convolutional	8	ReLU	–	3	same
2	MaxPooling	–	–	0.5	2	same
3	Convolutional	8	ReLU	–	3	same
4	MaxPooling	–	–	0.5	2	same
5	Dense	64	ReLU	0.5	–	–
6	Dense	64	ReLU	0.5	–	–
7	Dense	1	Sigmoid	–	–	–

All of the neural network models are trained in 500 training epochs with batch size equaling 1024. Early stopping and model checkpoints based on the validation loss are also used to prevent overfitting so that we can get the best-performing model on the validation set.

Prediction with Threshold In contrast to other application domains, in finance, a good trading strategy does not require its decision to be accurate at all times. It is usually acceptable (even preferable, factoring in the transaction costs) for a model to be sparse in making decisions, only generating "high conviction" calls, if this results in greater accuracy (Ghoshal and Roberts, 2020). We try to incorporate this idea by adding a confidence threshold of α to the probability output for all of the selected models. The threshold α is chosen so that it yields the highest precision within the validation set.

$$\alpha = \underset{\alpha \in A}{\arg\max}\, Precision(\alpha, Validation\ set) \tag{12}$$

After the threshold α has been chosen so that all models will have a similar range of recall, the other metrics such as precision, Sharpe ratio, and average return will be compared to find out the best performing model.

3.4. Model evaluation

Precision and Recall

Precision and recall are also popular metrics that usually go together for evaluating classifier performance and will be used as the main metrics in this paper.

The increases in either precision or recall are often achieved at the expense of the other, i.e. high precision is achieved at the expense of recall, and high recall is achieved at the expense of precision. Usually, an ideal model would love to have both high recall and high precision but this is usually hard to be achieved in practice. Due to the fact that a buy signal does not necessarily lead to a good return and investors usually look for a good return rather than a normal return, in combination with the existence of transaction costs, a good trading signal system generally prefers a high precision with a reasonable recall.

Return and Average Return

As this study focuses on the feasibility of trading strategy based on several signal generators, it is always a good idea to include the financial metrics such as return and risk-adjusted return into the study. For the sake of simplicity, we do not incorporate the transaction cost into the study. The return, therefore, will be calculated based on the compounded return formula:

$$Total\ Return = \prod_{i=1}^{N}(1 + R_i S_i) \tag{13}$$

Table 8
Performance of all long-only models.

Model	Input type	Feature type	Strategy	Validation				Test			
				Sharpe ratio	Precision	Recall	Avg return	Sharpe ratio	Precision	Recall	Avg return
GRU	candle	extended	long	**9.62**	**61.74%**	10.44%	**0.17%**	**1.85**	**58.39%**	10.87%	**0.04%**
GRU	candle	raw	long	7.13	58.86%	11.27%	0.13%	1.55	58.34%	11.57%	0.03%
GRU	ohlc	extended	long	3.58	58.08%	11.63%	0.08%	0.94	57.11%	10.76%	0.02%
CNN	ohlc	extended	long	2.29	57.82%	10.31%	0.08%	−0.75	57.02%	11.85%	−0.02%
LSTM	ohlc	raw	long	4.37	58.87%	10.96%	0.10%	−0.78	56.72%	12.40%	−0.02%
MLP	candle	extended	long	6.02	59.82%	10.47%	0.11%	0.46	56.03%	11.82%	0.01%
MLP	candle	raw	long	5.10	60.69%	10.20%	0.12%	1.57	55.90%	10.85%	0.04%
Logistic	candle	extended	long	0.23	59.00%	9.73%	0.10%	0.23	55.83%	9.30%	0.01%
CNN	candle	extended	long	2.48	58.74%	9.60%	0.09%	−0.89	55.81%	9.56%	−0.03%
CNN	candle	raw	long	4.14	57.93%	11.89%	0.12%	0.38	55.71%	12.08%	0.01%
MLP	ohlc	raw	long	2.29	57.56%	11.05%	0.07%	0.00	55.68%	11.57%	−0.02%
LSTM	candle	raw	long	9.17	60.20%	10.87%	0.15%	−0.62	55.64%	10.39%	−0.02%
LGBM	candle	raw	long	2.02	56.51%	11.72%	0.06%	−0.49	55.62%	11.92%	−0.01%
CNN	ohlc	raw	long	1.66	56.93%	9.44%	0.07%	0.02	55.60%	11.59%	0.00%
Logistic	ohlc	extended	long	2.69	57.94%	9.93%	0.07%	−0.69	55.53%	10.11%	−0.02%
GRU	ohlc	raw	long	3.46	58.06%	10.13%	0.08%	−0.39	55.19%	12.68%	−0.01%
Logistic	candle	raw	long	6.17	59.85%	9.08%	0.14%	−0.56	55.19%	8.98%	−0.02%
LGBM	candle	extended	long	6.99	60.03%	9.28%	0.14%	−0.20	55.12%	9.09%	0.00%
LGBM	ohlc	extended	long	6.61	59.74%	11.49%	0.12%	0.37	54.89%	10.64%	0.01%
LGBM	ohlc	raw	long	6.52	61.72%	9.75%	0.13%	−0.38	54.67%	9.49%	−0.01%
XGBoost	candle	raw	long	2.92	57.14%	10.18%	0.08%	−0.39	54.59%	10.04%	−0.01%
MLP	ohlc	extended	long	2.68	59.04%	11.00%	0.08%	−0.12	54.38%	11.06%	−0.01%
LSTM	candle	extended	long	7.55	60.88%	9.24%	0.13%	−1.18	54.29%	8.79%	−0.04%
LSTM	ohlc	extended	long	3.78	58.80%	10.73%	0.07%	−0.76	54.21%	11.78%	−0.02%
XGBoost	ohlc	raw	long	3.20	54.91%	9.48%	0.08%	1.17	54.10%	10.39%	0.03%
Logistic	ohlc	raw	long	3.07	59.04%	9.04%	0.08%	−0.78	54.09%	9.19%	−0.02%
XGBoost	ohlc	extended	long	1.88	56.97%	9.48%	0.05%	−0.53	52.64%	9.23%	−0.01%
XGBoost	candle	extended	long	3.53	57.48%	9.26%	0.08%	0.11	52.20%	8.24%	0.01%

where R_i is the return of every time step, and S_i is the binary signal generated by the model at each specific time step.

Using this total return as a metric, however, does not incorporate the transaction cost, and therefore the higher return may come from many small trades instead of the quality of every single trade (again recall versus precision). As a result, in this study, we will primarily use the metrics of average return and risk-adjusted return (Sharpe ratio) to reflect the quality of the signal instead.

$$Average\ Return = \frac{Total\ Return}{\sum_{i=1}^{N} S_i} \tag{14}$$

$$Sharpe\ Ratio = \frac{Average\ Return}{Standard\ Deviation\ of\ Return} \tag{15}$$

Significance Tests

In order to evaluate the relative performance of various model parameters, we not only rely on the previously discussed metrics but also conduct significance tests to determine if there are any significant differences between groups of models based on different features or algorithms. To accomplish this, we apply a two-sample t-test to the return of each group of models using the block bootstrapping method where we divide the data into n/k non-overlapping blocks. n is the sample size and k is the length of each block. k is chosen to be equal to $n^{1/3}$ as it is the optimal length proposed by Shalizi (2013). Each block is then sampled with replacement to recreate the original sample sets for 10,000 iterations. As the return samples of each model will have the same size, all of them will be divided by the same number of blocks with the same timestamps. For each iteration, the same block will be chosen then across different strategies.

4. Result and discussion

4.1. General discussion

As discussed in the previous section, the goal of our study is not only to find out the potential application of machine learning in generating trading signals but also to find out whether or not the representation of candlesticks and other popular technical indicators is necessary for the improvement of the models. Therefore, we applied several subsets of features compared to each other to compare their usefulness.

The first dimension for the set of features is the representation of the candlestick. In Section 3.1, we have mentioned that the candlestick has 3 main characteristics: the candle body, the upper shadow, and the lower shadow. In combination with the closing price, we use them to create the "candle" features dataset to compare with the traditional "ohlc" dataset, which contains open, high, low, and closing price, to see which approach serve better as the input for the machine learning models.

The second dimension is about the inclusion of additional technical indicators or not. The feature dataset with the inclusion of additional technical indicators is called the "extended" feature dataset. For the original set of features (only 4 candlestick features, trades, volume, and the cyclical), we called it the "raw" feature dataset.

Table 9
Performance of all short-only models.

Model	Input type	Feature type	Strategy	Validation				Test			
				Sharpe ratio	Precision	Recall	Avg return	Sharpe ratio	Precision	Recall	Avg return
LGBM	ohlc	raw	short	−0.51	54.00%	11.23%	−0.02%	**2.25**	**60.05%**	11.97%	**0.07%**
MLP	candle	extended	short	0.92	57.93%	9.19%	**0.04%**	1.45	59.16%	9.31%	0.04%
LGBM	candle	extended	short	**1.10**	55.59%	9.33%	**0.04%**	1.62	59.02%	9.36%	0.05%
LGBM	candle	raw	short	−0.60	54.01%	11.35%	−0.02%	1.08	58.77%	11.35%	0.03%
GRU	ohlc	raw	short	−1.54	55.92%	11.78%	−0.04%	1.45	58.31%	10.59%	0.05%
Logistic	ohlc	extended	short	0.61	54.91%	10.35%	0.02%	1.02	58.02%	10.75%	0.03%
LSTM	candle	raw	short	0.69	**59.55%**	11.40%	0.01%	0.66	57.84%	9.54%	0.01%
MLP	ohlc	extended	short	0.48	55.24%	10.14%	0.01%	−0.05	57.77%	12.08%	0.00%
LSTM	ohlc	raw	short	0.37	57.54%	11.32%	0.00%	0.46	57.66%	10.16%	0.01%
LGBM	ohlc	extended	short	0.48	53.98%	11.92%	0.01%	0.45	57.66%	11.28%	0.01%
Logistic	candle	extended	short	0.33	56.32%	9.09%	0.01%	0.68	57.63%	8.90%	0.02%
LSTM	ohlc	extended	short	0.32	56.43%	10.94%	0.00%	−0.35	57.45%	10.94%	−0.01%
CNN	ohlc	raw	short	0.77	55.86%	10.75%	0.02%	1.00	57.29%	11.32%	0.03%
GRU	candle	raw	short	2.02	57.29%	9.05%	−0.05%	1.20	57.28%	8.19%	0.03%
Logistic	ohlc	raw	short	−0.88	53.76%	10.85%	−0.03%	0.86	57.18%	10.75%	0.02%
CNN	candle	extended	short	1.19	56.20%	9.90%	0.03%	1.17	57.10%	9.20%	0.03%
LSTM	candle	extended	short	−0.24	55.01%	11.87%	−0.01%	−0.13	56.52%	10.91%	−0.01%
GRU	ohlc	extended	short	−2.60	54.05%	45.35%	−0.06%	1.03	56.15%	46.05%	0.03%
MLP	candle	raw	short	−0.54	54.44%	10.04%	−0.04%	0.60	56.02%	9.15%	0.00%
XGBoost	candle	raw	short	0.11	52.96%	10.19%	0.00%	1.69	55.92%	10.48%	0.05%
CNN	ohlc	extended	short	−0.94	52.17%	44.80%	−0.03%	0.46	55.89%	44.75%	0.02%
GRU	candle	extended	short	−0.38	57.09%	46.11%	−0.01%	0.79	55.84%	47.91%	0.02%
XGBoost	candle	extended	short	−0.72	52.22%	10.35%	−0.03%	0.73	55.78%	10.27%	0.02%
CNN	candle	raw	short	−0.85	55.29%	9.92%	−0.02%	−0.12	55.42%	9.36%	−0.01%
MLP	ohlc	raw	short	0.16	56.72%	10.83%	0.00%	−1.15	54.55%	10.43%	−0.01%
Logistic	candle	raw	short	0.08	51.95%	47.20%	0.00%	1.34	54.44%	47.24%	0.02%
XGBoost	ohlc	extended	short	−0.54	53.08%	11.44%	−0.02%	0.36	54.04%	10.87%	0.01%
XGBoost	ohlc	raw	short	−1.40	52.31%	9.43%	−0.04%	0.03	53.02%	9.45%	0.00%

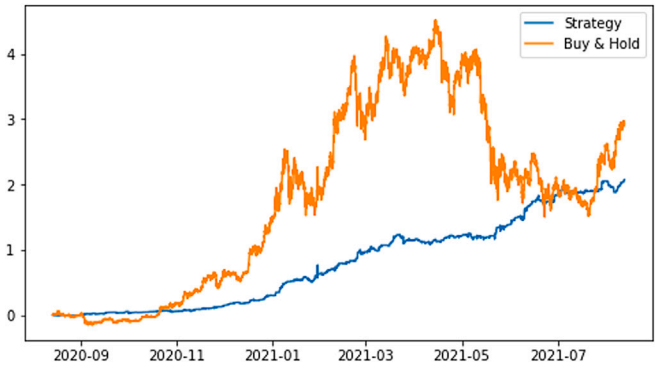

Fig. 3. Model performance in validation set.

In combination with the primary two dimensions of long and short strategy, in the end, each type of machine learning model are fitted 8 different times as the combination of the 3 dimensions. In the end, we have in total 56 different models and all of their results are shown in Tables 8 and 9.

The two tables show the list of all models with their performance metrics calculated on the validation and the test set. The order in which they are listed is based on their precision on the test set, from the highest to the lowest. From just a quick view of the two tables, we can see that most of the top models are deep learning models with the exception of the LightGBM in shorting strategy. Additionally, all of the machine learning models overall outperform almost all of the candlestick patterns in both precision and average return, given the same range of recall. Given the fact that the market is quite bearish in the test set and the shorting is generally more difficult in practice, we believe the performance for the long-only strategy is more reliable compared to the short-only strategy

The best-performing model here in both validation and test set is the GRU long-only model which is trained on the extended candle dataset with 58.36% precision and the Sharpe ratio of 1.85. The performance of this strategy is shown in Figs. 3 and 4 where it is compared with the buy-and-hold strategy of Bitcoin for reference.

From the two figures, we can see the model outperformed the benchmark and there are way fewer downtrends and drawdowns during both periods. The cumulative return of the strategy also increases steadily and does not come from only a few single trades or some specific period.

Fig. 4. Model performance in test set.

Table 10
Comparison between 2 candlestick representation methods.

Data	Validation				Test			
	Sharpe ratio	Precision	Recall	Avg return	Sharpe ratio	Precision	Recall	Avg return
candle	2.72	57.31%	12.79%	0.055%	0.52	56.26%	12.67%	0.012%
ohlc	1.53	56.48%	13.06%	0.036%	0.17	55.96%	13.36%	0.005%

Table 11
Comparison between the raw dataset and extended dataset.

Data	Validation				Test			
	Sharpe ratio	Precision	Recall	Avg return	Sharpe ratio	Precision	Recall	Avg return
extended	2.14	57.01%	14.05%	0.048%	0.27	56.12%	14.13%	0.007%
raw	2.11	56.78%	11.80%	0.043%	0.42	56.10%	11.90%	0.009%

4.2. Best candlestick features

Table 10 presents the average performance metrics of all machine learning models using either the "ohlc" features or the "candle" features. From the table, we can see that the performance of the candle data is slightly better than the performance of the OHLC data (0.8% increase in precision for the validation set and only 0.3% for the test set). The average Sharpe ratio and return from the models that used the candlestick representation are also higher than that of the models that used the OHLC representation. However, as we performed a bootstrap t-test on the return of the two sets, we did not find the difference in return to be statistically significant (p-value=0.125).

4.3. Inclusion of technical indicators

The difference caused by the addition of technical indicators is even less noticeable compared to the previous case. Table 11 illustrates the performance of the extended features in comparison with the raw features.

The addition of 12 new technical indicators does not substantially increase the performance of all machine learning models at all. The inclusion of those indicators only led to an increase of a measly 0.02% in precision for the test set and even decrease the financial performance in the Sharpe ratio and the average return. There is almost no significant difference between the mean return between the two groups as a bootstrap t-test also returns the p-value of 0.394. We also observe an increase in the number of signals with more than 2% improvements in recall in both the validation and test dataset. However, the reason for this 2% increase is due to the fact that there are four short models with more than 40% recall and three of them used the "extended" features. This phenomenon only happens to the short models because both our validation set and test set are in a bearish market and it would be easy for the model to overtrade and still have good results.

4.4. Models comparison

Finally, we would like to compare the general performance of every machine learning model that we used. Table 12 shows the average performance metrics of all models.

From the table, it is easy to observe that the recurrent neural networks (GRU and LSTM) are better in both precision and average return for this specific classification task with the exception of the LightGBM. They are ranked from the highest to the lowest in

Table 12
Average Performance of all models.

Data	Validation				Test			
	Sharpe ratio	Precision	Recall	Avg return	Sharpe ratio	Precision	Recall	Avg return
GRU	2.66	57.64%	19.47%	0.037%	1.05	57.08%	19.83%	0.025%
LGBM	2.83	56.95%	10.76%	0.056%	0.53	56.97%	10.64%	0.018%
LSTM	3.25	58.41%	10.92%	0.059%	−0.34	56.29%	10.61%	−0.011%
CNN	1.34	56.37%	14.58%	0.045%	0.16	56.23%	14.96%	0.003%
MLP	2.14	57.68%	10.36%	0.048%	0.34	56.19%	10.78%	0.008%
Logistic	1.54	56.60%	14.41%	0.048%	0.26	55.99%	14.40%	0.005%
XGBoost	1.12	54.63%	9.98%	0.026%	0.40	54.03%	9.87%	0.011%

the precision calculated on the test set. Among them, the GRU model is better in almost every metric compared to other models. To verify if GRU is better than other models, we also performed a two-sample bootstrap t-test for the returns of the GRU models and the returns of all other models that are not GRU. The result shows a significant difference between the mean of returns of the two groups at a 90% confidence interval (p-value=0.0753). The LightGBM models, come second but only perform well for the short-only strategy in the bearish market. Their long-only versions are quite worse compared to the others. As a result, we do not value their performance that highly. Regarding the LSTM model, despite its high precision, its financial performance is not that great compared to other models. Given that their performance on the validation set is the highest, it is likely that the LSTM models suffered from overfitting in the training phase. Compared to the GRU, the LSTM tends to have more parameters (3 gates instead of 2 gates) given the same level of model complexity. This can give the LSTM an edge in capturing more complex patterns in the data but it can also make the LSTM more prone to overfitting. Lowering the model complexity and performing hyperparameters tuning can definitely solve this problem. As for the CNN models, recent literature has suggested that they can outperform RNNs in sequential modeling (Ghoshal and Roberts, 2020; Bai et al., 2018). However, in this case, the state-of-the-art CNNs could not outperform other architectures. It is likely due to the fact that the long lookback period (24 h) of the time series seems to favor the ability to retain long-range information about the RNNs over the ability to identify local patterns of the CNNs. The logistic model's simplicity results in lower precision than most models, but it outperforms XGBoost. The XGBoost, despite its similar nature with the LightGBM, actually performed way worse than all other models in precision, with 2% precision lower than the 2nd worst one. However, its financial metrics such as the Sharpe ratio are similar to those of the LightGBM models.

5. Conclusion and future work

5.1. Conclusion

Our analysis has pointed out that it is not inconceivable that the Bitcoin price history might contain some predictive information. That information can even be enhanced through the use of several popular technical indicators so that they can be exploited by quantitative methods to create a profitable trading strategy. Our detailed results show that our prediction models using the machine learning approach trained on the Binance cryptocurrency exchange can create profits that are better than the classic Buy-and-Hold strategy and provide much better precision than random guesses.

Our study has shown how several popular machine learning methods such as neural networks, decision trees, and logistic regression performed on Bitcoin price trend prediction. The results turned out that the neural networks did the best compared to other models, especially in the case of fewer features. However, among several neural network architectures, the state-of-the-art CNNs did not outperform other architectures with comparable complexity as we expected (Ghoshal and Roberts, 2020). The recurrent neural network, on the other hand, shows superior performance with the GRU architectures and, to a lesser extent, the LSTM architectures. Between the two, the GRU performed better than LSTM. However, due to the limited sample size, we cannot arbitrarily conclude that the GRU is better than the LSTM.

5.2. Future work

In the future, there are many things that are worth trying to improve the performance of our system:

1. **Hyperparameters tuning:**
   In our study, we mostly used a predefined set of hyperparameters and architectures suggested by other studies for our machine learning models. However, these hyperparameters are definitely not the optimal ones. Due to our limitation in time and training system specification, we could not do this step but fine-tuning a machine learning system, especially in the case of neural network models, can significantly improve its performance.
2. **More feature engineering and feature selection**:
   Only a few technical indicators are chosen as the features in this study. In practice, there are a plethora of other technical indicators that are yet to be tested. Aside from technical indicators, there can be other sources of data such as fundamental data, and alternative data like news, Twitter, etc. The inclusion of new features and their selection can definitely improve the performance of our system.

3. **Ensembling models**:
   In this study, we only take into account the performance of every single model. However, the method of the ensemble can also be applied to other machine learning models like neural networks. Robert, S. et al. (2020), in their work, mentioned an improvement in their system using models ensembling.
4. **Incorporation of transaction cost:**
   The omission of transaction costs in our performance evaluation is perhaps the most apparent flaw of our study. Transaction costs can be difficult to model as they depend on the bid–ask spreads, the slippage, the volume of the executed transaction, and various other aspects. A more thorough incorporation of the transaction cost could have been more beneficial for the validation of the results.
5. **Testing on other markets:**
   Despite that our predicting systems may work with the Bitcoin market, it does not mean they will work with other financial assets. More tests should be made in other markets to verify the robustness of the models.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

References

Anghel, D.-G., 2021. A reality check on trading rule performance in the cryptocurrency market: Machine learning vs. technical analysis. Finance Res. Lett. 39, 101655.

Bai, S., Kolter, J.Z., Koltun, V., 2018. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv preprint arXiv:1803.01271.

Basak, S., Kar, S., Saha, S., Khaidem, L., Dey, S.R., 2019. Predicting the direction of stock market prices using tree-based classifiers. North Am. J. Econ. Finance 47, 552–567.

Bianchi, D., Büchner, M., Tamoni, A., 2021. Bond risk premiums with machine learning. Rev. Financ. Stud. 34 (2), 1046–1089.

Cosenza, D.N., Korhonen, L., Maltamo, M., Packalen, P., Strunk, J.L., Næsset, E., Gobakken, T., Soares, P., Tomé, M., 2021. Comparison of linear regression, k-nearest neighbour and random forest methods in airborne laser-scanning-based prediction of growing stock. Forestry: Int. J. Forest Res. 94 (2), 311–323.

Ghoshal, S., Roberts, S., 2020. Thresholded ConvNet ensembles: Neural networks for technical forecasting. Neural Comput. Appl. http://dx.doi.org/10.1007/s00521-020-04877-9.

Gu, S., Kelly, B., Xiu, D., 2020. Empirical asset pricing via machine learning. Rev. Financ. Stud. 33 (5), 2223–2273.

Hancock, J., Khoshgoftaar, T.M., 2020. Performance of catboost and xgboost in medicare fraud detection. In: 2020 19th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 572–579.

Nobre, J., Neves, R.F., 2019. Combining principal component analysis, discrete wavelet transform and XGBoost to trade in the financial markets. Expert Syst. Appl. 125, 181–194.

Rodriguez, P.N., Rodriguez, A., 2004. Predicting stock market indices movements. WIT Trans. Model. Simul. 38.

Shalizi, C., 2013. Advanced Data Analysis from an Elementary Point of View. Citeseer.

Wang, Z., Oates, T., et al., 2015. Encoding time series as images for visual inspection and classification using tiled convolutional neural networks. In: Workshops At the Twenty-Ninth AAAI Conference on Artificial Intelligence, Vol. 1. AAAI Menlo Park, CA, USA.

Yun, K.K., Yoon, S.W., Won, D., 2021. Prediction of stock price direction using a hybrid GA-XGBoost algorithm with a three-stage feature engineering process. Expert Syst. Appl. 186, 115716.

Zhang, Y., Chu, G., Shen, D., 2021. The role of investor attention in predicting stock prices: The long short-term memory networks perspective. Finance Res. Lett. 38, 101484.

Zolotareva, E., 2021. Aiding long-term investment decisions with xgboost machine learning model. In: International Conference on Artificial Intelligence and Soft Computing. Springer, pp. 414–427.